

Hashing pelo método de *folding*

O **hashing** é um mecanismo usado para garantir a integridade da informação transmitida através das redes de comunicação. Por exemplo, o fornecedor de um programa, por vezes, disponibiliza igualmente um **hash** (condensado) ou **digest** (resumo) do ficheiro original para ser comparado com o calculado a partir do ficheiro recebido por *download*. Se não forem idênticos, significa que durante a transmissão o conteúdo do ficheiro foi accidental ou intencionalmente modificado.

Sendo a dimensão (nº de caracteres ou de bytes) do **hash** muito inferior à da informação a partir da qual é calculado, os métodos de **hashing** terão que minimizar a probabilidade de ficheiros distintos produzirem **hashes** idênticos. São conhecidos vários métodos de **hashing**, MD5 (Message Digest) e SHA (Secure Hash Algorithms), entre outros, que garantem aquela propriedade.

O método **folding** (dobragem) proposto para implementação de **hashing**, pela sua simplicidade, não garante a propriedade acima referida, mas constitui um bom exercício de programação.

No método **folding**, como em qualquer outro, é calculado um **hash** com um comprimento (nº de caracteres ou de bytes) fixo a partir de conjuntos de informações de dimensões variáveis. Um texto, por exemplo, será dividido em blocos ou linhas de igual comprimento que serão “adicionados” constituindo o resultado final o **hash** pretendido. Para efetuar a “adição” cada carácter será substituído pelo respetivo nº de ordem em ASCII.

Aplicando este método ao texto abaixo para cálculo de um **hash** de comprimento igual a cinco

Hashing por folding

Começamos por obter, com recurso à função **ord()**, os números de código ASCII de cada um dos caracteres

Números ASCII: 72 97 115 104 105 110 103 32 112 111 114 32
70 111 108 100 105 110 103

Em seguida, formam-se as seguintes linhas com cinco números de código ASCII

Hashi (72 97 115 104 105)
ng po (110 103 32 112 111)
r fol (114 32 70 111 108)
ding5 (100 105 110 103 53)

Serão acrescentados caracteres (números de código) ao texto para que o nº total seja múltiplo do comprimento do **hash** a calcular. Neste exemplo foi usado o algarismo 5 (ASCII 53), o comprimento do **hash**.

072	097	115	104	105	
110	103	032	112	111	
114	032	070	111	108	
100	105	110	103	053	
396	337	327	430	377	(adição)
141	82	72	175	122	(resto da divisão por 255)
8D	52	48	AF	7A	(caracteres hexadecimais)

Com o resultado da adição de cada coluna calcula-se o resto da divisão por 256 (nº total de caracteres codificados em ASCII). Cada número obtido deverá ser convertido para o sistema hexadecimal porque alguns códigos ASCII não têm representação tipográfica.

O “keyed hash” (hash com chave) é um mecanismo usado para comprovar a autenticidade (autoria) de um texto recorrendo a uma chave secreta. Uma sequência de números entre 0 e 255, de comprimento igual ao do **hash**, será “adicionada” ao valor obtido por aplicação do **folding**, seguindo as mesmas regras (resto da divisão por 256) obtendo-se uma “assinatura” do texto. Esta assinatura garante não só a integridade dos dados, mas igualmente a autenticidade, ou seja, que os dados não foram modificados comprova a respetiva autoria.

Os dados e a correspondente assinatura são transmitidos no mesmo ficheiro e quando forem rececionados e calculada uma assinatura que será comparada com recebida. Se forem idênticas, estarão asseguradas a integridade e autenticidade dos dados.